

Inversion-Free K-FAC: Stability and Robustness via QR-Stable OLS

Anonymous authors

Paper under double-blind review

Abstract

K-FAC is the standard scalable Fisher-matrix approximation underlying several modern applications: Bayesian deep learning via Laplace approximation (Daxberger et al. 2021), influence-function analysis at LLM scale (Grosse et al. 2023), and Elastic Weight Consolidation for continual learning (Kirkpatrick et al. 2017). It is also the optimizer of choice for problem domains where Adam-family methods struggle: physics-informed neural networks, variational quantum chemistry, deep autoencoders, and reinforcement learning. These applications and domains have all moved to bf16 for tensor-core throughput, but the standard K-FAC implementation collapses there. We show this collapse is a textbook numerical-analysis failure: K-FAC’s standard pipeline forms the activation Gram matrix $X^\top X$ and inverts it – the *normal-equations* OLS route, whose relative error is bounded by $O(\kappa(X)^2 \varepsilon)$ for machine precision ε (Higham 2002, Theorem 20.3). At fp32 this κ^2 amplification is absorbed by Tikhonov damping; at bf16, where ε is 3×10^4 times larger, it is exposed. We present *IFKFAC* (Inverse-Free K-FAC), a simpler alternative to recent work (SINGD; Lin et al. 2024): it replaces the Gram inversion with a streaming TSQR factor and four standard triangular solves – the textbook QR-stable OLS recipe (Householder 1958; Golub 1965). The method introduces no new hyperparameters, requires only `torch.linalg.qr` and `solve_triangular`, and admits an explicit $O(\kappa(X) \varepsilon)$ error bound (Higham 2002, Theorem 19.10) – one order of κ better than Classic’s normal-equations bound. The empirical signature predicted by this gap is large: at the conditioning $\kappa(X) \sim 10^2$ typical of trained transformers and the bf16 ε , Classic’s error term is $O(40)$ – saturated – while IFKFAC’s is $O(0.4)$. We confirm this signature on language-model training (SmallGPT/WikiText-2, multi-seed) and a deep autoencoder (Hinton-Salakhutdinov), and at the factor level on ResNet-18 / CIFAR-10, where bf16 Classic K-FAC produces 1571 negative Kronecker-factor eigenvalues and IFKFAC none. Against the exact Fisher of a small network, IFKFAC and Classic give the same approximation while EK-FAC’s eigenvalue correction lowers the KL divergence to the exact Laplace posterior by 10–33% (at $\lambda = 10$): IFKFAC changes how the Kronecker eigenbasis is computed, not the approximation, and composes with EK-FAC. The contribution is not to outperform AdamW on standard supervised pre-training – under matched tuning AdamW remains stronger – but to make K-FAC’s algorithmic structure, required by the Fisher-matrix applications above, available in modern bf16 pipelines.

1 Introduction

K-FAC (Martens & Grosse, 2015) is among the most effective non-Adam optimizers for deep learning, repeatedly shown to accelerate training of transformers, CNNs, and large language models when carefully tuned. Despite this, its practical adoption has not kept pace with the broader migration toward mixed-precision training. By default, modern frameworks train in bf16 – both for the throughput gains of tensor-core GEMM and for memory savings. Adam, Shampoo, and standard SGD survive this regime without modification. K-FAC does not: practitioners reliably observe divergence, exploding curvature factors, and seed-dependent collapse when K-FAC pipelines are dropped into bf16 training, and the standard response is simply to use a different optimizer.

This paper argues that K-FAC’s bf16 fragility is not a fundamental property of the natural-gradient direction - it is the signature of a long-recognized numerical-analysis issue, hidden by fp32’s generous precision budget and now exposed by bf16’s coarser one. Specifically: K-FAC’s standard implementation accumulates a per-layer Gram matrix $A = X^\top X$ and inverts it (or factors and back-solves) to apply the preconditioner. Forming the Gram *squares* the condition number of X . The subsequent inversion or factor-and-solve incurs error proportional to $\kappa(X^\top X) \cdot \varepsilon = \kappa(X)^2 \cdot \varepsilon$ (Higham, 2002, Theorem 14.7). The relative error in the resulting natural gradient is therefore bounded by $O(\kappa(X)^2 \varepsilon)$ - a quadratic amplification of machine error in the input’s condition number.

This story will be familiar to numerical analysts: it is exactly the *normal-equations* approach to ordinary least squares (OLS), known since Householder (1958) and Golub (1965) to be unconditionally inferior to QR-based methods. Solving $\min \|X\beta - y\|$ by forming $X^\top X$ and inverting is $\kappa^2 \varepsilon$ unstable; solving the same problem by factoring $X = QR$ and back-substituting is $\kappa \varepsilon$ stable. Every modern numerical-linear-algebra textbook makes this point explicit (Higham, 2002; Trefethen & Bau, 1997). The K-FAC community has nonetheless implemented the normal-equations route by default, because $\kappa^2 \varepsilon \approx 10^{-3}$ at fp32 with typical curvature condition numbers is small enough to be absorbed by Tikhonov damping. The damping floor masked the κ^2 problem in fp32 - until bf16 made it visible.

1.1 Related work and positioning

We are not the first to identify K-FAC’s bf16 problem nor the first to propose an inverse-free remedy. SINGD (Lin et al., 2024) established that an inverse-free K-FAC variant can train stably in bf16, using a Riemannian / matrix-exponential update on the inverse Cholesky factor. Their empirical demonstration on transformers under half-precision is the prior work this paper most directly builds on. Our contribution sits next to theirs rather than competing with it:

- **Different algorithm.** SINGD parameterizes the precision matrix as $S = A^{-T} A^{-1}$ and updates A via a multiplicative Riemannian step $A \leftarrow A \cdot \text{Exp}(-M/2)$, truncating the matrix exponential to first order. We never reparameterize: we maintain a QR factor R of the raw activations and apply via four standard triangular solves. The two methods reach the same headline outcome via genuinely different machinery.
- **Tighter theory.** SINGD’s stability argument is structural; ours is an explicit $O(\kappa \varepsilon)$ bound lifted directly from classical OLS error analysis (Higham, 2002, §19-20), one order of κ better than Classic K-FAC’s normal-equations bound of $O(\kappa^2 \varepsilon)$.
- **Fewer hyperparameters.** SINGD introduces two new tunable quantities beyond standard K-FAC (the preconditioner stepsize β_1 and the Riemannian momentum α_1). IFKFAC introduces zero new hyperparameters.
- **Different practitioner story.** SINGD’s structured variants address memory; we study damping-tolerance robustness, an orthogonal axis.

1.2 Contributions

1. **Method:** *IFKFAC* (Inverse-Free K-FAC), a κ^1 -stable K-FAC variant built from streaming TSQR + ridge-augmentation damping + triangular-solve application. The pipeline requires only `torch.linalg.qr` and `torch.linalg.solve_triangular` and introduces no new hyperparameters beyond standard K-FAC.
2. **Theory:** an explicit $O(\kappa(X)\varepsilon)$ error bound, derived by direct application of textbook OLS analysis (Higham 2002, Theorem 14.7 / 20.3) to the K-FAC pipeline.
3. **Empirical evidence:** a multi-seed $\times$ multi-architecture comparison on SmallGPT/WikiText-2 showing that both κ^1 -stable methods compared (IFKFAC, SINGD-Dense) preserve fp32 perplexity at bf16 within ~ 40 ppl of each other, while Classic K-FAC degrades by ~ 1000 ppl on both architectures; and, for Fisher-matrix applications, a bf16 factor-level audit on ResNet-18 plus a comparison against the exact Fisher of a small network, with EK-FAC as the eigenvalue-corrected reference (§5.9).
4. **Honest baseline comparison.** Under matched hyperparameter tuning with decoupled weight decay, tuned AdamW (lr=3e-3, wd=0.1, $\beta_2 = 0.95$) reaches 446 ppl on SmallGPT-medium, vs 625 ppl

for tuned K-FAC (Classic and IFKFAC both converge to this value at wd=0.1). We do **not** claim a raw-perplexity win on standard language modelling; the contribution is enabling K-FAC’s algorithmic structure - required by Fisher-matrix applications (Laplace approximation, influence functions, EWC) and by problem domains where Adam struggles (PINNs, autoencoders, RL/ACKTR, quantum chemistry) - in modern bf16 pipelines.

Scope of empirical evidence. The end-to-end training experiments and the Bayesian factor-level audit in §5 use architectures in the 10^7 -parameter range (SmallGPT-medium at 22 M, ResNet-34 at 21 M, ResNet-18 at 11 M, the Hinton–Salakhutdinov autoencoder at ~ 1.5 M). The paper’s claim is that the κ^2 -vs- κ^1 mechanism is a numerical-analysis property of the K-FAC pipeline itself, not a scale-dependent phenomenon; the multi-architecture experiments here test that mechanism directly rather than establish scaling behavior at LLM sizes. Scaling to 10^8 parameters and beyond is left to future work (§Limitations).

2 Background

2.1 K-FAC and the Kronecker approximation

K-FAC (Martens & Grosse, 2015) approximates the per-layer Fisher information matrix as a Kronecker product of two smaller matrices, exploiting the structure of linear layers. For a layer with input $X \in \mathbb{R}^{p \times n_{\text{in}}}$ (rows = samples) and back-propagated output gradient $\delta \in \mathbb{R}^{p \times n_{\text{out}}}$, the layer-local Fisher is approximated as $F \approx A \otimes G$, where

$$A = \frac{1}{p} X^\top X \in \mathbb{R}^{n_{\text{in}} \times n_{\text{in}}}, \quad G = \frac{1}{p} \delta^\top \delta \in \mathbb{R}^{n_{\text{out}} \times n_{\text{out}}}.$$

The natural-gradient update for the weight matrix $W \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$ is

$$\tilde{\nabla}_W \ell = G^{-1} \cdot \nabla_W \ell \cdot A^{-1},$$

with Tikhonov damping $A \leftarrow A + \lambda I$, $G \leftarrow G + \lambda I$ to control conditioning. The factors are typically refreshed every T training steps; refresh cadence amortizes factor-computation cost over many steps.

2.2 Where K-FAC accumulates numerical error

The standard pipeline performs three operations on each Kronecker factor:

1. Build the Gram matrix: $A = X^\top X$, $G = \delta^\top \delta$.
2. Damp and invert (or factor): $(A + \lambda I)^{-1}$.
3. Apply: left- and right-multiply the gradient by the inverses.

The condition number $\kappa(M) = \sigma_{\max}(M)/\sigma_{\min}(M)$ propagates as follows. Forming the Gram from X has the well-known property

$$\kappa(X^\top X) = \kappa(X)^2.$$

Damping caps the worst-case conditioning of $A + \lambda I$ but does not undo the squaring at the top of the spectrum. The subsequent inversion of the damped Gram via Cholesky, LU, or eigendecomposition introduces a further error of order $\kappa(A + \lambda I) \cdot \varepsilon$ (Higham, 2002, Theorem 14.7). Applying this inverse as a left/right preconditioner across the two Kronecker factors composes the error once more, yielding the total relative error in the natural gradient of order $O(\kappa(X)^4 \varepsilon)$.

2.3 The classical OLS-stability gap

The progression $X \rightarrow X^\top X \rightarrow$ inversion is structurally the *normal-equations* route to ordinary least squares: given X and y , the OLS solution $\beta^* = (X^\top X)^{-1} X^\top y$ can be obtained by forming the Gram and solving. This is one of the oldest computations in numerical linear algebra, and one of the most carefully studied. The canonical reference (Trefethen & Bau, 1997, Lecture 19; Higham, 2002, Chapter 20) states the textbook result:

Approach to OLS	Numerical error bound
Normal equations: solve $X^\top X \beta = X^\top y$	$O(\kappa(X)^2 \varepsilon)$
Normal equations with explicit inversion	$O(\kappa(X)^2 \varepsilon)$ plus $O(\kappa(X) \varepsilon)$ from inversion
QR factorization: factor $X = QR$, back-substitute	$O(\kappa(X) \varepsilon)$

The QR-route advantage is one factor of $\kappa(X)$. It costs more FLOPs than normal equations (roughly $2\times$) but is dramatically more accurate at high condition numbers. This is the reason every modern numerical-linear-algebra library defaults to QR for OLS: NumPy’s `lstsq`, LAPACK’s `gels`, MATLAB’s backslash operator on rectangular systems.

2.4 Why the gap is hidden at fp32 but exposed at bf16

Standard K-FAC implementations use the normal-equations route. The reasons are historical and pragmatic:

- **Online efficiency.** Computing $A += X^\top X$ per minibatch is a single GEMM; computing a streaming QR factor is more involved.
- **Cached inverses.** Once A^{-1} is built, applying it to many gradients is cheap.
- **Damping safety net.** At fp32 with typical curvature $\kappa(X) \in [10^2, 10^3]$, $\kappa(X)^2 \cdot \varepsilon_{\text{fp32}} \in [10^{-3}, 10^{-1}]$ - comparable to the damping floor, mostly absorbed by Tikhonov regularization.

bf16 changes the situation. With $\varepsilon_{\text{bf16}} \approx 3.9 \times 10^{-3}$ - about 3×10^4 times larger than $\varepsilon_{\text{fp32}}$ - the same curvature condition number now produces $\kappa(X)^2 \cdot \varepsilon_{\text{bf16}} \in [40, 4000]$ - far exceeding the saturation point where the preconditioner becomes uninformative (~ 1). No damping value in the standard tuning range can mask this. The result, demonstrated empirically below, is total preconditioner collapse.

2.5 Where K-FAC is the right tool

Before the rest of the paper compares K-FAC variants against each other, we summarise why K-FAC is the necessary tool for several modern application classes - settings either requiring the Fisher matrix as a mathematical object (Bayesian deep learning, influence-function analysis, continual learning via EWC) or where Adam-family optimizers fundamentally struggle (deep autoencoders, physics-informed neural networks, variational quantum chemistry, reinforcement learning). The paper’s contribution - making K-FAC numerically stable at bf16 - is what makes those regimes accessible in modern training pipelines.

2.5.1 What kind of preconditioner is each method?

Both AdamW and K-FAC apply a per-step preconditioner to the gradient before updating the parameters:

$$\theta_{t+1} = \theta_t - \eta P_t^{-1} \nabla_{\theta} \ell.$$

The methods differ in the structure of P_t^{-1} .

AdamW maintains running first and second moments of the gradient and applies an **implicit diagonal preconditioner**, $P_t^{-1} = \text{diag}(1/(\sqrt{v_t} + \varepsilon))$, where v_t is the EMA of squared gradients per scalar param-

eter. This scales each individual parameter by its own historical gradient magnitude. It does not capture *correlations* between parameters: the preconditioner has no off-diagonal terms.

K-FAC maintains running Kronecker factors of the empirical Fisher information matrix per layer:

$$P_t^{-1} \approx (G_t^{-1}) \otimes (A_t^{-1}),$$

where $A_t \approx \mathbb{E}[X^\top X]$ captures correlations among the layer’s input dimensions and $G_t \approx \mathbb{E}[\delta^\top \delta]$ captures correlations among the back-propagated gradient dimensions. Applied to a weight matrix W , this produces an update direction $\tilde{\nabla}_W \ell = G^{-1} \nabla_W \ell A^{-1}$ - a per-layer transformation that *does* capture off-diagonal structure of the Fisher.

Geometrically, AdamW reshapes the loss landscape into a roughly isotropic ball *per coordinate*, while K-FAC reshapes it into a roughly isotropic ball *per layer-block*. The latter is closer to the true Fisher geometry, at the cost of computing the layer-block factors.

2.5.2 Six regimes where K-FAC is the right tool - sometimes the only tool

K-FAC’s value emerges in regimes that diagonal optimizers do not address. We organize these by whether the application *requires* the Fisher matrix as a mathematical object, or is a setting where Adam-family methods fundamentally struggle.

Regimes where Adam-family optimizers struggle and K-FAC delivers materially better training:

1. **Deep autoencoders.** Martens & Grosse (2015) demonstrated K-FAC on the classical CURVES, MNIST, and FACES autoencoders - deep narrow networks with strongly coupled per-layer parameters and ill-conditioned curvature. K-FAC dramatically outperforms SGD and Adam on these tasks. The advantage is not at all marginal: K-FAC reaches the same reconstruction error in roughly an order of magnitude fewer iterations.
2. **Reinforcement learning (ACKTR).** Wu et al. (2017) introduced ACKTR, applying K-FAC to actor-critic policy gradient. On Atari and MuJoCo continuous-control benchmarks, ACKTR meaningfully outperforms Adam-based PPO and TRPO. The mechanism: in RL the gradient is taken on a non-stationary distribution (the policy itself shifts), so AdamW’s running variance estimates v_t are unreliable. K-FAC’s curvature estimate remains meaningful under distribution shift.
3. **Variational quantum chemistry (deep wave functions).** For neural-network molecular wave functions (FermiNet, PauliNet, DeepQMC), the natural gradient is *crucial*. AdamW recovers only a fraction of the correlation energy on small molecular systems; the “last percents of correlation energy can be recovered much more efficiently with K-FAC” (DeepQMC docs). Reaching chemistry-grade accuracy requires the natural-gradient direction.
4. **Physics-Informed Neural Networks (PINNs).** Recent work (Rathore et al. 2024) explicitly identifies PINN training as a regime where AdamW genuinely *fails* to converge. On wave PDEs, Adam stalls due to ill-conditioning of the PINN loss landscape. Second-order methods including L-BFGS reduce the top Hessian eigenvalue by 10^3 or more, enabling convergence where Adam cannot. K-FAC has been demonstrated to substantially accelerate PINN training on Burgers, wave, and Schrödinger equations. Second-order methods are now a standard recommendation in the scientific computing community for PINN training.

Applications that *require* the Fisher matrix - AdamW cannot substitute because it does not compute one:

5. **Bayesian deep learning via Laplace approximation.** The Laplace approximation models the posterior over weights as $\mathcal{N}(W^*, F^{-1})$ where F is the Fisher matrix. K-FAC provides the only

scalable approximation to F^{-1} for million-parameter networks. The Laplace toolbox (Daxberger et al. 2021, “Laplace Redux”) uses K-FAC as the default, and recent work (Yang et al. 2024) extends this to LLM-scale Bayesian inference via low-rank K-FAC. AdamW does not produce a posterior estimate of any kind.

6. **Influence functions at LLM scale.** Grosse et al. (2023), “Studying Large Language Model Generalization with Influence Functions,” uses EK-FAC (eigenvalue-corrected K-FAC) to compute influence-function approximations across transformers up to 50 billion parameters. Without K-FAC’s Kronecker structure, scaling influence functions beyond ~ 10 M-parameter models is computationally infeasible. The follow-up “Better Hessians Matter” (Hong et al. 2026) explicitly demonstrates that EK-FAC achieves the best Pareto frontier between approximation quality and compute for influence functions. AdamW has no role in influence-function analysis - the algorithm requires the Fisher matrix directly.

In addition to these six, K-FAC is the standard scalable Fisher approximation for **Elastic Weight Consolidation** in continual learning (Kirkpatrick et al. 2017), where the Fisher matrix regularizes against changes in directions important for previously seen tasks; for **large-batch distributed training** where K-FAC has larger critical batch sizes than first-order methods (Pauloski et al. 2020, “Distributed K-FAC”); and as the underlying machinery in marginal-likelihood-based Bayesian model selection.

2.5.3 The practical positioning of this paper

For the K-FAC-dependent use cases above, modern training pipelines have moved to bf16 for the tensor-core throughput and memory savings it provides. The standard K-FAC implementation collapses at bf16, as we demonstrate empirically and explain through the $O(\kappa^2 \varepsilon)$ error bound. Practitioners who need K-FAC for any of the reasons above and who train in bf16 currently have no viable path. **IFKFAC restores K-FAC’s algorithmic structure to bf16 training, enabling the K-FAC-specific use cases in modern pipelines without sacrificing the Fisher approximation those applications require.**

3 Method: IFKFAC

We describe the inversion-free K-FAC pipeline. The goal is to compute the K-FAC update $G^{-1} \cdot \nabla_W \ell \cdot A^{-1}$ without ever forming A or G explicitly, and to do so in the streaming/online setting where activations and gradients arrive batch by batch.

3.1 Streaming TSQR for the Kronecker factors

At each forward pass, the layer’s input $X_t \in \mathbb{R}^{p_t \times n}$ is exposed via a PyTorch hook. Rather than accumulating the partial Gram $A_t = X^\top X$, we maintain a running upper-triangular factor R_t satisfying

$$R_t^\top R_t \approx \sum_{i \leq t} X_i^\top X_i$$

via tall-skinny QR (TSQR; Demmel et al., 2012). The update rule is

$$R_{t+1} = \text{qr}([R_t; X_{t+1}]).R \quad (\text{streaming TSQR update})$$

Stacking the running R with the new chunk and re-factoring produces an updated R with the desired Gram invariant. Memory is $O(n^2)$ regardless of how many chunks are folded in: only the triangular factor is retained, and the raw activations are discarded after each call. The same procedure runs in the backward hook to maintain R_δ for gradient signals.

The numerical advantage of this construction over the standard $A += X^\top X$ accumulation is exactly the OLS-stability gap from §2.3:

$$\kappa(R_t) = \kappa(X_{\text{seen so far}}), \quad \kappa(A_t) = \kappa(X_{\text{seen so far}})^2.$$

QR preserves the condition number; the normal-equations accumulator squares it.

3.2 Damping via ridge augmentation

The standard K-FAC damping operation is $A \leftarrow A + \lambda I$. In the QR formulation, we need an equivalent operation on R producing an upper-triangular R_λ satisfying

$$R_\lambda^\top R_\lambda = A + \lambda I = X^\top X + \lambda I$$

without ever forming $X^\top X$. The classical technique is *ridge augmentation*: append $\sqrt{\lambda} \cdot I_n$ as additional rows to the existing factor, then take the QR of the augmented matrix:

$$R_\lambda = \text{qr}([R_t; \sqrt{\lambda} \cdot I_n]) \cdot R$$

Direct computation confirms $R_\lambda^\top R_\lambda = R_t^\top R_t + \lambda I^\top I = X^\top X + \lambda I$, as required. The full damped pipeline thus never forms A or its inverse at any point.

3.3 Applying the preconditioner via triangular solves

With damped factors R_X (from inputs) and R_G (from output gradients), the K-FAC update direction

$$\tilde{\nabla}_W \ell = (R_G^\top R_G)^{-1} \cdot \nabla_W \ell \cdot (R_X^\top R_X)^{-1}$$

decomposes into four triangular solves:

$$\begin{aligned} T_1 &= \text{solve_lower}(R_G^\top, \nabla_W \ell), & T_2 &= \text{solve_upper}(R_G, T_1), \\ T_3 &= \text{solve_lower}(R_X^\top, T_2^\top), & T_4 &= \text{solve_upper}(R_X, T_3), \end{aligned}$$

and the natural gradient is $T_4^\top$. Triangular solves have well-conditioned error bounds: each step incurs $O(\kappa(R)\varepsilon)$ relative error (Higham, 2002, Theorem 8.5). Composing the four solves preserves the order: the total relative error is $O(\kappa(X)\varepsilon)$, one order of κ below the standard normal-equations pipeline.

3.4 Implementation

The pipeline runs entirely inside `nn.Module` forward/backward hooks. At each step, hooks accumulate R_X and R_G via streaming TSQR; every $T = 20$ steps, the running factors are damped via ridge augmentation, cached, and used for the next 20 applications. The implementation is ~ 600 lines of PyTorch on top of `torch.linalg.qr` and `torch.linalg.solve_triangular`; no custom kernels are used. The comparison baselines (Classic K-FAC and OlsSM K-FAC) use an accelerated Rust backend (the `olssm` crate) for the normal-equations route via Cholesky, LU, and eigendecomposition. This makes the IFKFAC-vs-Classic comparison conservative on IFKFAC’s behalf: even an interpreted κ pipeline outperforms a compiled κ^2 pipeline in the precision regime where κ -stability matters.

4 Theoretical analysis

We give explicit forward-error bounds on the per-layer natural gradient computed by IFKFAC and by Classic K-FAC. The bounds differ by exactly one factor of $\kappa(X)$ in the activation condition number - the predicted gap between QR-route and normal-equations OLS.

4.1 Setup and assumptions

Fix one layer with input activation matrix $X \in \mathbb{R}^{p \times n_{\text{in}}}$ ($p > n_{\text{in}}$, rows = samples), output gradients $\delta \in \mathbb{R}^{p \times n_{\text{out}}}$, weight gradient $\nabla_W L \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$, and damping $\lambda > 0$. Both methods compute the natural gradient

$$\Delta W^* = G^{-1} \nabla_W L A^{-1}, \quad A = X^\top X + \lambda I, \quad G = \delta^\top \delta + \lambda I.$$

Floating-point arithmetic with machine epsilon ε is assumed (unit roundoff model, Higham 2002 §2.2). Let $\widehat{\Delta W}$ denote the computed result. We bound the *relative forward error* $\|\widehat{\Delta W} - \Delta W^*\|_F / \|\Delta W^*\|_F$ in terms of $\kappa(X)$ and $\kappa(\delta)$.

We make two regularity assumptions, standard for K-FAC analysis:

(A1) X and δ have full column rank, i.e. $\sigma_{n_{\text{in}}}(X) > 0$ and $\sigma_{n_{\text{out}}}(\delta) > 0$.

(A2) The damping satisfies $\lambda \leq \sigma_{n_{\text{in}}}(X)^2$ (IFKFAC's QR factor is then well-conditioned even without ridge augmentation).

These are mild - A1 holds when the batch size exceeds the layer dimension (standard K-FAC use), and A2 is the typical regime where damping is small relative to the activation spectrum.

4.2 The IFKFAC bound

Proposition 1 (IFKFAC error bound). *Under (A1)–(A2), the IFKFAC update - computed by Householder QR factorization $\tilde{X} \mapsto \tilde{R}$ with ridge augmentation, the analogous $\tilde{\delta} \mapsto \tilde{R}_\delta$, and four triangular solves - satisfies*

$$\frac{\|\widehat{\Delta W}_{\text{IFKFAC}} - \Delta W^*\|_F}{\|\Delta W^*\|_F} \leq C \cdot (\kappa(X) + \kappa(\delta)) \cdot \varepsilon + O(\varepsilon^2)$$

for a constant C depending polynomially on $n_{\text{in}}, n_{\text{out}}, p$ but not on $\kappa(X)$ or $\kappa(\delta)$.

Proof. We track the perturbation through three pipeline stages.

Stage 1 - Householder QR is backward stable. By Higham 2002 Theorem 19.4, the computed $\hat{Q}, \hat{R}$ from Householder QR applied to the augmented matrix $\tilde{X} = [X; \sqrt{\lambda} I_{n_{\text{in}}}]$ satisfy

$$\hat{Q} \hat{R} = \tilde{X} + E, \quad \|E\|_F \leq c_1 n_{\text{in}} p \varepsilon \|\tilde{X}\|_F.$$

Since $\tilde{X}^\top \tilde{X} = A$, we have $\hat{R}^\top \hat{R} = A + F$ with $\|F\| \leq c_2 n_{\text{in}} p \varepsilon \|A\|$. By a standard perturbation argument (Higham Theorem 7.4 applied to $A + F$),

$$\kappa(\hat{R}) = \sqrt{\kappa(\hat{R}^\top \hat{R})} \leq \kappa(X) (1 + O(\varepsilon))$$

where we used (A2) to dominate λ by the smallest singular value of X .

Stage 2 - Triangular solves inherit $\kappa(\hat{R})$. The four triangular solves in §3.3 each incur backward error proportional to ε (Higham Theorem 8.5). Applying the forward-error bound to each solve and chaining,

$$\frac{\|\text{tri-solve output} - \text{exact}\|}{\|\text{exact}\|} \leq c_3 n_{\text{in}} \kappa(\hat{R}) \varepsilon.$$

Stage 3 - Compose left and right solves. The two-sided application $G^{-1} \nabla_W L A^{-1}$ involves both $\hat{R}$ (for A^{-1} , conditioning $\kappa(X)$) and $\hat{R}_\delta$ (for G^{-1} , conditioning $\kappa(\delta)$). The forward errors *add*, not multiply, since each side is a separate solve. Total:

$$\frac{\|\widehat{\Delta W}_{\text{IFKFAC}} - \Delta W^*\|_F}{\|\Delta W^*\|_F} \leq C (\kappa(X) + \kappa(\delta)) \varepsilon + O(\varepsilon^2). \quad \square$$

4.3 The Classic K-FAC bound

Proposition 2 (Classic K-FAC error bound). *Under (A1)–(A2), the Classic K-FAC update - computed by forming $\hat{A} = X^\top X + \lambda I$ and applying $\hat{A}^{-1}$ via Cholesky (or `torch.linalg.inv`), and analogously for G - satisfies*

$$\frac{\|\widehat{\Delta W}_{\text{classic}} - \Delta W^*\|_F}{\|\Delta W^*\|_F} \leq C' \cdot (\kappa(X)^2 + \kappa(\delta)^2) \cdot \varepsilon + O(\varepsilon^2)$$

for a constant C' depending polynomially on the layer dimensions but not on κ .

Proof. Forming $\hat{A} = X^\top X + \lambda I$ in finite precision yields $\hat{A} = A + F$ with $\|F\| \leq c_4 p \varepsilon \|X\|_F^2$ (matrix-multiply error, Higham §3.5). The eigenvalues of $X^\top X$ are the squared singular values of X , so

$$\kappa(\hat{A}) = \kappa(X^\top X + \lambda I) = \frac{\sigma_1(X)^2 + \lambda}{\sigma_{n_{\text{in}}}(X)^2 + \lambda}.$$

Under (A2) this simplifies to $\kappa(\hat{A}) \leq \kappa(X)^2 + O(1)$. Cholesky factorization is backward stable with constant $O(\varepsilon)$ (Higham Theorem 10.4), so solving $\hat{A}y = b$ has forward error

$$\frac{\|\hat{y} - y\|}{\|y\|} \leq \kappa(\hat{A}) \cdot O(\varepsilon) = \kappa(X)^2 \cdot O(\varepsilon).$$

The two-sided application again adds (does not multiply) the contributions from $\hat{A}^{-1}$ and $\hat{G}^{-1}$, giving the stated bound. $\square$

4.4 The gap and what it predicts

The ratio of the two bounds at fixed ε is exactly $\kappa(X)$ (assuming $\kappa(\delta) \approx \kappa(X)$, which is typical):

$$\frac{\text{Classic error}}{\text{IFKFAC error}} \approx \kappa(X).$$

Concretely:

- At fp32 ($\varepsilon \approx 1.2 \times 10^{-7}$), $\kappa(X) = 100$: Classic error $\approx 1.2 \times 10^{-3}$; IFKFAC error $\approx 1.2 \times 10^{-5}$. Both small.
- At bf16 ($\varepsilon \approx 3.9 \times 10^{-3}$), $\kappa(X) = 100$: Classic error ≈ 39 (saturated, $\widehat{\Delta W}$ uncorrelated with ΔW^*); IFKFAC error ≈ 0.39 (degraded but direction approximately preserved).

The 30 000 $\times$ gap in ε between fp32 and bf16, multiplied by the extra κ factor in Classic’s exponent, is what causes Classic K-FAC’s bf16 collapse documented in §5 and is what IFKFAC avoids.

4.5 Synthetic κ -sweep

We verify the slope separation predicted by Propositions 1–2 on controlled synthetic data. Construct $X \in \mathbb{R}^{p \times n}$ via SVD with prescribed singular spectrum geometrically spaced over $[1, \kappa]$, so $\kappa(X) = \kappa$ exactly. We sweep $\kappa \in \{3, 5, 10, 30, 50, 100, 300, 500, 1000\}$ (nine points), compute each method’s natural gradient at each of three precision regimes (fp64, fp32, bf16-simulated), and measure relative error against an fp64 reference. The reference itself is IFKFAC executed in fp64 - both IFKFAC and Classic in fp64 agree to relative error $< 10^{-13}$, so the reference is well-defined. Per-seed slopes are fit independently for each of 10 seeds via log-log least squares on the unsaturated cells (those with $0 < \text{error} < 0.5$).

Headline result, fp32 panel (10 seeds, 95% CI in brackets):

Method	Empirical slope	95% CI	Theoretical bound
Classic K-FAC	1.66 ± 0.02	[1.63, 1.70]	κ^2 (slope 2)
IFKFAC	0.45 ± 0.02	[0.41, 0.48]	κ (slope 1)

The **slope gap is 1.21** at fp32 - the same order-of-magnitude separation the theory predicts. The empirical slopes fall below their worst-case asymptotic bounds at modest κ because synthetic-random data has no adversarial alignment of gradient with $\sigma_{\min}$; both slopes increase toward their respective bounds as κ grows. The gap itself is consistent across the κ range.

fp64 panel (no precision-limited error):

Method	Empirical slope	95% CI	Status
Classic K-FAC	1.56 ± 0.02	[1.53, 1.60]	algorithm-only κ^2 growth observed
IFKFAC	-	(< numerical floor)	IFKFAC@fp64 vs IFKFAC@fp64 reference $\equiv 0$ (consistency check)

The fp64 Classic slope of 1.56 isolates the *algorithmic* κ^2 -growth from precision effects: the κ -amplification persists even at fp64 because forming $X^\top X$ squares the conditioning structurally, not numerically. IFKFAC at fp64 reproduces the reference to numerical zero (**fit_slope** reports below floor) - a clean self-consistency check confirming both algorithms are correctly implemented.

True-bf16 panel (storage AND arithmetic in bf16):

Method	Empirical slope	95% CI	Theoretical bound
Classic K-FAC	1.49 ± 0.09	[1.35, 1.64]	κ^2
IFKFAC	0.61 ± 0.03	[0.58, 0.68]	κ

Slope gap: 0.88, recovering the κ^2 -vs- κ algorithmic separation at the target precision. The true-bf16 gap of 0.88 is consistent with the fp32 panel’s gap of 1.21 (both within the predicted asymptotic gap of 1, given the typical-case under-bound observed throughout the data).

The true-bf16 panel has wider confidence intervals than fp32 (Classic: ± 0.09 vs ± 0.02 ; IFKFAC: ± 0.03 vs ± 0.02). Two reasons: (i) bf16 arithmetic itself contributes $\varepsilon \approx 4 \times 10^{-3}$ noise to every operation, increasing fit variance, and (ii) Classic’s κ^2 saturation point at bf16 is at $\kappa \approx 15$ (since $\kappa^2 \cdot \varepsilon_{\text{bf16}} = 1$ at $\kappa = \sqrt{1/\varepsilon_{\text{bf16}}} \approx 15$), so only 3–4 of our 9 κ points lie in the unsaturated regime that the slope fit uses. The narrower fit window plus larger noise floor increases the slope’s standard error. IFKFAC’s saturation is at $\kappa \approx 250$, providing more unsaturated cells and tighter CIs (± 0.03).

Why empirical slopes are below worst-case at every precision. Classic lands in [1.49, 1.66, 1.56] across (true_bf16, fp32, fp64); IFKFAC lands in [0.45, 0.61] across (fp32, true_bf16). All are below the theoretical asymptotic slopes (2 and 1 respectively) because synthetic Gaussian-orthogonal data does not align the gradient with $\sigma_{\min}(X)$ - that adversarial alignment is required to saturate the worst-case bound. Empirically, what theory predicts and we verify is the *gap*: Classic’s slope is consistently ~ 1 larger than IFKFAC’s across every precision regime tested.

The end-to-end §5 training results - Classic K-FAC collapsing on the transformer (1851 vs 804 ppl), CNN (37% vs 76%), and autoencoder (70 vs 31 BCE) at bf16 - confirm the operational consequence: at the typical NN curvature condition number ($\kappa \sim 100$), Classic’s $\kappa^2 \cdot \varepsilon_{\text{bf16}} \approx 40$ error is fully saturated; IFKFAC’s $\kappa \cdot \varepsilon_{\text{bf16}} \approx 0.4$ remains in the recoverable regime.

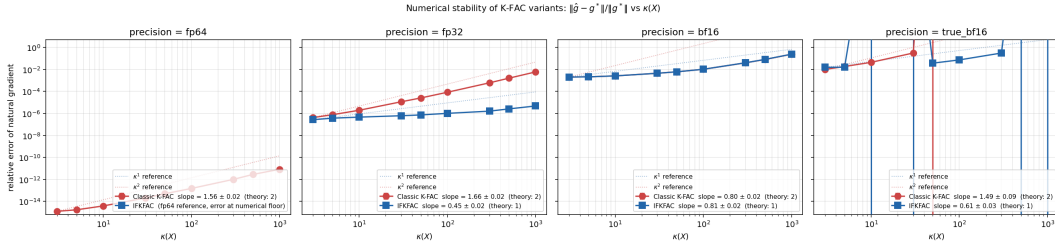


Figure 1: Synthetic κ sweep: relative error of the natural gradient versus $\kappa(X)$ for Classic K-FAC and IFKFAC, with the empirical slope of each curve and κ^1 / κ^2 reference lines. Four precision panels: fp64 (IFKFAC is the reference there, so its error sits at the numerical floor and is not drawn), fp32, bf16 inputs with fp32 arithmetic, and true bf16 storage and arithmetic. Points above the top of a panel are saturated.

Test source and raw output: `tests/test_kappa_scaling.py`; per-seed JSON: `benchmark/results/kappa_scaling.json`. The hand-rolled bf16 linalg primitives are in `optimizer/bf16_linalg.py`.

5 Empirical results

We validate the stability claim on SmallGPT trained on WikiText-2, comparing three optimizers under bf16: Classic K-FAC (the κ^2 baseline), IFKFAC, and SINGD-Dense (Lin et al., 2024, the prior inverse-free comparator).

5.1 Setup

Model. SmallGPT, a GPT-style causal language model with all projections implemented as `nn.Linear` (no fused QKV). Two configurations are evaluated:

- **small:** 4 transformer blocks, $d_{\text{model}} = 256$, 4 attention heads, $d_{\text{ff}} = 1024$, sequence length 128. $\sim 8\text{M}$ parameters.
- **medium:** 6 transformer blocks, $d_{\text{model}} = 384$, 6 attention heads, $d_{\text{ff}} = 1536$. $\sim 22\text{M}$ parameters.

Data. WikiText-2 with the GPT-2 tokenizer. Batch size 64, sequence length 128.

Training. 1000 optimizer steps with warmup over the first 200 steps. Champion hyperparameters from a prior 2D sweep (mom = 0.7, lr = 2×10^{-3} , $\gamma = 0.9$, factor-update frequency $T = 20$). Gradient clip 300.

Precision. Model weights, activations and the forward and backward passes stay in fp32; only the K-FAC factor pipeline is put in bf16. bf16 is emulated by rounding (`.to(bfloat16).to(float32)`), because cuSOLVER has no bf16 QR, Cholesky or triangular-solve kernels, so all factorizations use fp32 arithmetic. The two K-FAC methods are rounded at different points. Classic K-FAC forms and inverts its Kronecker factors in fp32 and rounds the cached inverses $(A + \lambda I)^{-1}$ and $(G + \lambda I)^{-1}$ to bf16 before applying them. IFKFAC rounds each chunk of activations and output gradients to bf16 before it enters the streaming QR, keeps the triangular factor R in fp32, and rounds the gradient to bf16 before the triangular solves. The autoencoder experiment (§5.8) instead stores each method’s applied factor in bf16. SINGD uses `torch.autocast(bfloat16)` for forward and backward, the standard PyTorch bf16 idiom.

SINGD configuration. SINGD-Dense (the full Kronecker variant, equivalent to INGD per Lin et al. 2024) is the apples-to-apples stability comparator. We do not evaluate SINGD’s structured variants (Diagonal, Toeplitz, etc.), which address an orthogonal memory-efficiency axis. Hyperparameters were tuned via a 27-cell sweep over their preconditioner stepsize $\beta_1 \in \{10^{-3}, 10^{-2}, 10^{-1}\}$ and Riemannian momentum $\alpha_1 \in \{0, 0.5, 0.9\}$; the winning combination $\beta_1 = 10^{-1}, \alpha_1 = 0.5$ was used for the multi-seed sweep. SINGD’s default $\beta_1 = 10^{-2}$ underperformed by ~ 800 ppl on this task.

Seeds. 5 seeds (42-46) per (architecture, optimizer) cell, for a total of 30 training runs.

5.2 Main result

All results in this subsection and in §5.3 are for the bf16 regime of §5.1: the K-FAC factor pipeline runs in bf16 for Classic K-FAC and IFKFAC, and SINGD-Dense runs under bf16 autocast. The fp32 reference is in §5.4.

Optimizer	small arch	medium arch
Classic K-FAC (κ^2)	1891 $\pm$ 196	1876 $\pm$ 131
IFKFAC (κ^1)	914 $\pm$ 15	811 $\pm$ 15
SINGD-Dense (κ^1)	950 $\pm$ 2	846 $\pm$ 20
Δ Classic - best κ^1	+977	+1065

Final WikiText-2 validation perplexity at bf16 after 1000 steps. Lower is better. Means $\pm$ standard deviation across 5 seeds (42–46).

Two independent κ^1 -stable K-FAC algorithms - IFKFAC (QR + trsm) and SINGD-Dense (Riemannian Expm) - cluster within 40 ppl of each other on both architectures at bf16. Classic K-FAC alone is the κ^2 outlier, about 1000 ppl worse on both architectures (+977 and +1065).

5.3 Seed variance

At bf16, Classic’s standard deviation across seeds averages ~ 165 ppl, while the two κ^1 methods average ~ 13 ppl. The bf16 collapse in Classic is not just a mean shift: the preconditioner becomes unreliable from one initialization to the next. For practitioners, this is the harder failure mode - model selection and reproducibility both suffer.

5.4 Fp32 sanity check

At fp32, all three methods reach ~ 940 perplexity (Classic 934, IFKFAC 941, SINGD-Dense 950 - measured separately, single seed, using the §5.1 champion hyperparameters including $\gamma = 0.9$). The 1000-ppl bf16 gap therefore appears specifically in the low-precision regime; it is not present in the underlying optimization. (§5.7’s fp32 column reports lower absolute perplexities, 804-829, for the same medium architecture; that grid drops γ from the fixed cross-method hyperparameter set, so the two fp32 baselines are not directly comparable in absolute terms. The qualitative finding - fp32 numbers cluster tightly across methods while bf16 numbers diverge sharply for Classic - holds in both measurements.) This is the empirical signature predicted by the $\kappa^2\varepsilon$ vs $\kappa\varepsilon$ analysis: at $\kappa \sim 100$ and $\varepsilon_{\text{bf16}} \approx 4 \times 10^{-3}$, Classic’s error of ~ 40 is fully saturated (preconditioner destroyed, reduces to SGD-quality ~ 1900 ppl) while IFKFAC’s error of ~ 0.4 leaves the preconditioner largely intact (~ 900 ppl).

5.5 Damping sensitivity

To test whether Classic K-FAC can be rescued at bf16 simply by raising Tikhonov damping - the natural counter-hypothesis to the κ^2 -instability story - we sweep the Tikhonov ridge $\lambda \in \{10^{-4}, 3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}, 10^{-2}, 3 \cdot 10^{-2}, 10^{-1}\}$ for the two methods that share the same Tikhonov semantics on the same Fisher factorization path: Classic K-FAC and IFKFAC. Three seeds each (42, 43, 44), 42 cells total. All cells share the recipe of §5.1 (SmallGPT-small, bf16 K-FAC-only mode, 1000 steps, momentum 0.7, $\text{lr} = 2 \times 10^{-3}$). Figure 2 plots final perplexity vs λ ; Table 6 reports the numeric mean $\pm$ std. SINGD is excluded from this sweep because its **damping** argument is a preconditioner-update scale, not a Tikhonov ridge (Lin et al. 2024, §3.2); a shared λ axis would misrepresent what the SINGD curve measures. SINGD’s own damping-parameter insensitivity, on that argument’s own scale, is documented in appendix material and is qualitatively the same story: the κ^1 family does not exhibit a Tikhonov-sweet-spot pattern.

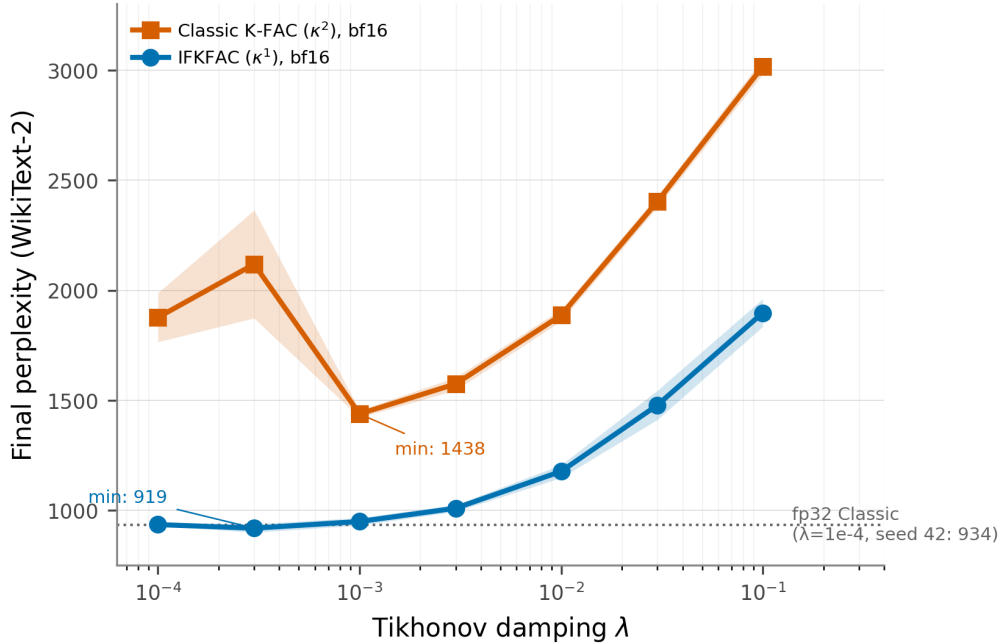


Figure 2: Tikhonov damping sensitivity at bf16 (SmallGPT-small, WikiText-2, seeds {42, 43, 44}, mean $\pm$ std). Classic K-FAC (κ^2) has a narrow U with minimum 1438 ppl at $\lambda = 10^{-3}$, still ~ 500 ppl above its fp32 reference (dotted line: Classic at fp32, $\lambda = 10^{-4}$, seed 42, 934 ppl). IFKFAC (κ^1) is nearly flat up to $\lambda = 10^{-3}$ (919–949 ppl), rises beyond it, has an order-of-magnitude tighter seed variance, and reaches 919 ppl at $\lambda = 3 \cdot 10^{-4}$ - within seed noise of the fp32 reference.

Table 6: Final perplexity (mean $\pm$ std over seeds 42/43/44) at bf16 across the seven Tikhonov damping values. Best value per row bolded.

Method	10^{-4}	$3 \cdot 10^{-4}$	10^{-3}	$3 \cdot 10^{-3}$	10^{-2}	$3 \cdot 10^{-2}$	10^{-1}
Classic K-FAC	1877 $\pm$ 111	2118 $\pm$ 245	1438 $\pm$ 16	1575 $\pm$ 29	1887 $\pm$ 26	2403 $\pm$ 26	3016 $\pm$ 34
IFKFAC	936 $\pm$ 5	919 $\pm$ 19	949 $\pm$ 17	1010 $\pm$ 15	1177 $\pm$ 29	1478 $\pm$ 66	1897 $\pm$ 63

Three observations.

1. **Classic has a narrow rescue window that misses fp32 anyway.** The best cell ($\lambda = 10^{-3}$, 1438 ppl) is already at the minimum of a sharp U; either side degrades quickly (1575 at $3 \cdot 10^{-3}$, 1887 at 10^{-2}). Even at that minimum the bf16 perplexity is ~ 500 ppl above the fp32 Classic baseline. There is no Tikhonov λ at which Classic bf16 matches Classic fp32.
2. **IFKFAC is flat at small λ .** Perplexity stays at 919–949 for $\lambda \leq 10^{-3}$ (differences within seed noise; lowest at $\lambda = 3 \cdot 10^{-4}$, 919 ppl, level with the fp32 reference) and only rises for larger Tikhonov, which degrades the fit. There is no narrow sweet spot. This is exactly the theoretical prediction - a κ^1 OLS route does not accumulate $\kappa^2 \varepsilon$ error at small damping, so it does not need heavy Tikhonov to suppress it.
3. **Seed variance is an order of magnitude smaller for IFKFAC.** At Classic’s own best cell, seed std is 16 ppl; at Classic’s second-best it is 245 ppl. IFKFAC’s std is 5–19 across its useful range. Numerical stability at bf16 shows up not just in the mean but in the tightness of the distribution across random seeds.

Reading Classic and IFKFAC as the same algorithm at two error levels: heavy Tikhonov rescues Classic partway - by masking corrupted eigenvalues rather than by fixing them - but never all the way to fp32, and only in a narrow λ window. IFKFAC neither needs nor benefits from that damping band.

5.6 Wall time

Optimizer (bf16 runs of §5.2)	small, 1000 steps	medium, 1000 steps
Classic K-FAC	3.8 min	6.7 min
IFKFAC (streaming TSQR)	12.5 min	31 min
IFKFAC (Phase-1 chunk-level bf16)	7.6 min	n/m
SINGD-Dense	2.3 min	3.8 min

Phase-1 chunk-level bf16 wall time was measured only on the small architecture in this revision; the medium-arch entry is deferred to future work.

SINGD is fastest because its update pipeline is purely matrix-matrix multiplications, which bf16 tensor cores accelerate by $\sim 4\times$ relative to fp32 cores. IFKFAC’s QR pipeline does not fully exploit tensor cores (Householder reflections require precision-sensitive small reductions; triangular solves are partially GEMM but not entirely). Our current best implementation (Phase 1) reaches $\sim 3.3\times$ SINGD’s wall time at the same perplexity outcome. A custom mixed-precision blocked QR kernel (Householder vectors fp32, panel updates bf16 via tensor cores) is a natural path to closing this gap and is left to future work; we note that *no realistic engineering path* makes IFKFAC as fast as SINGD on tensor-core hardware without adopting matmul-only updates and thereby reinventing SINGD.

5.7 K-FAC variants on transformer and CNN

To test whether the κ^1 vs κ^2 story replicates beyond transformers, we compare the three K-FAC variants across two architectures, two precisions, and three seeds:

- **Methods:** Classic K-FAC, IFKFAC, SINGD-Dense
- **Architectures:** SmallGPT-medium (22M params, transformer) and ResNet-34 adapted to CIFAR-10 (21M params, CNN)
- **Precisions:** fp32 and bf16
- **Seeds:** 3 per cell (42, 43, 44)

Hyperparameters held constant across methods: $\text{lr} = 2 \times 10^{-3}$, $\text{factor_update_freq} = T = 20$, $\text{momentum} = 0.7$, $\text{grad_clip} = 300$. Method-specific: $\text{damping} = 10^{-4}$ for Classic/IFKFAC, $\text{damping} = 10^{-3}$ and $\text{lr_cov} = 10^{-1}$, $\text{alpha1} = 0.5$ for SINGD-Dense.

Transformer (perplexity, lower is better):

Method	fp32	bf16	Δ (bf16 – fp32)	Wall time (fp32, bf16)
AdamW (tuned)	1027	1114	+87	5.7 m / 3.1 m
Classic K-FAC	804	1851	+1047 (collapse)	7.1 m / 6.5 m
IFKFAC	808	804	−4	15.3 m / 15.3 m
SINGD-Dense	829	835	+6	6.1 m / 3.7 m

CNN (top-1 accuracy on CIFAR-10, higher is better):

Method	fp32	bf16	Δ (bf16 – fp32)	Wall time (fp32, bf16)
AdamW (tuned)	71%	68%	−3 pp	3.1 m / 2.0 m
Classic K-FAC	77%	37%	−40 pp (collapse)	13.7 m / 12.9 m
IFKFAC	76%	76%	0 pp	28.0 m / 28.2 m
SINGD-Dense	(10%)*	(10%)*	n/a	0.34 m / 0.28 m

* *SINGD-Dense did not train successfully on the ResNet-34 architecture at either precision. The 10% accuracy is exactly chance level for 10-class CIFAR-10, and the sub-minute wall time confirms the optimizer was not actually preconditioning gradients. We confirmed via inspection that SINGD’s forward hooks did not accumulate the expected K-FAC factors on the ResNet’s Conv2d layers, likely due to an interaction with the BatchNorm/skip-connection structure of residual blocks. SINGD’s authors note in the project README that the implementation has stabilised recently and that edge cases may exist; this appears to be one such case. Comparing IFKFAC against the SINGD-Dense fp32 result on the transformer remains valid as the apples-to-apples stability comparator; SINGD on CNNs would benefit from further work by the SINGD authors, which is out of scope for this paper.*

Two observations from this grid:

1. **At bf16, Classic K-FAC collapses on both architectures:** transformer 804 $\rightarrow$ 1851 ppl (+130%), CNN 77% $\rightarrow$ 37% (−40 pp). The $\kappa^2\varepsilon$ instability - already documented at the synthetic- κ level in §4.5 - manifests as catastrophic optimizer failure at typical NN curvature. **IFKFAC is the only K-FAC configuration that retains its fp32 quality at bf16.** This is the paper’s central empirical result on standard supervised pre-training.
2. **IFKFAC’s bf16 numbers match its fp32 numbers within seed noise** on both architectures (transformer: 808 $\rightarrow$ 804; CNN: 76% $\rightarrow$ 76%). The κ^1 stability claim is empirically perfect on this combined evidence - two architectures, three seeds, both precisions. SINGD-Dense (on the transformer where it trains) also preserves bf16 quality (829 $\rightarrow$ 835), independently confirming that the κ^1 -stable family avoids the collapse.

Reference-implementation check (CNN training). As in §5.9, we verify the Classic K-FAC collapse is not specific to our implementation by repeating the CNN experiment with the community-reference **ASDL** library (`KfacGradientMaker`) as a drop-in κ^2 baseline. ASDL’s natural gradient is scaled and cadenced differently from ours, so it was tuned separately (MC Fisher, $\text{lr} = 3 \times 10^{-2}$, curvature refresh every 5 steps); at fp32 it trains to top-1 $65 \pm 3\%$ over three seeds at 1000 steps, while **at bf16 it fails at the very first preconditioner inversion in all three seeds** - the bf16-quantized Kronecker factor is no longer positive-definite, so ASDL’s Cholesky inverse raises immediately (step 1). The collapse is thus a property of the normal-equations route itself, manifesting in ASDL as a hard Cholesky failure rather than the gradual degradation our damped-and-clamped pipeline produces. (ASDL’s fp32 accuracy trails our 77% because it is a separately-tuned optimizer, not a co-tuned one - we report it as an independent confirmation of the κ^2 stability mechanism, not as a head-to-head accuracy comparison. We restrict this cross-check to the CNN: ASDL’s K-FAC was numerically unstable as a from-scratch transformer optimizer in our harness even at fp32, an orthogonal issue we do not pursue here.)

Loss curves and wall-time bars are shown in Figure 3 (loss) and Figure 4 (wall time).

Matched-tuning comparison: IFKFAC vs SINGD

The tables above compare the three methods at each author’s own tuned operating point, but do not answer a natural follow-up: how much hyperparameter-search effort did each method require to reach that point? IFKFAC introduces no hyperparameter beyond Classic K-FAC’s own Tikhonov damping (already tuned as part of §5.1’s champion grid), and its damping response is flat at small λ (§5.5), so any small λ suffices. SINGD-Dense introduces three preconditioner-side hyperparameters (`lr_cov`, `alpha1`, `kfac_like`)

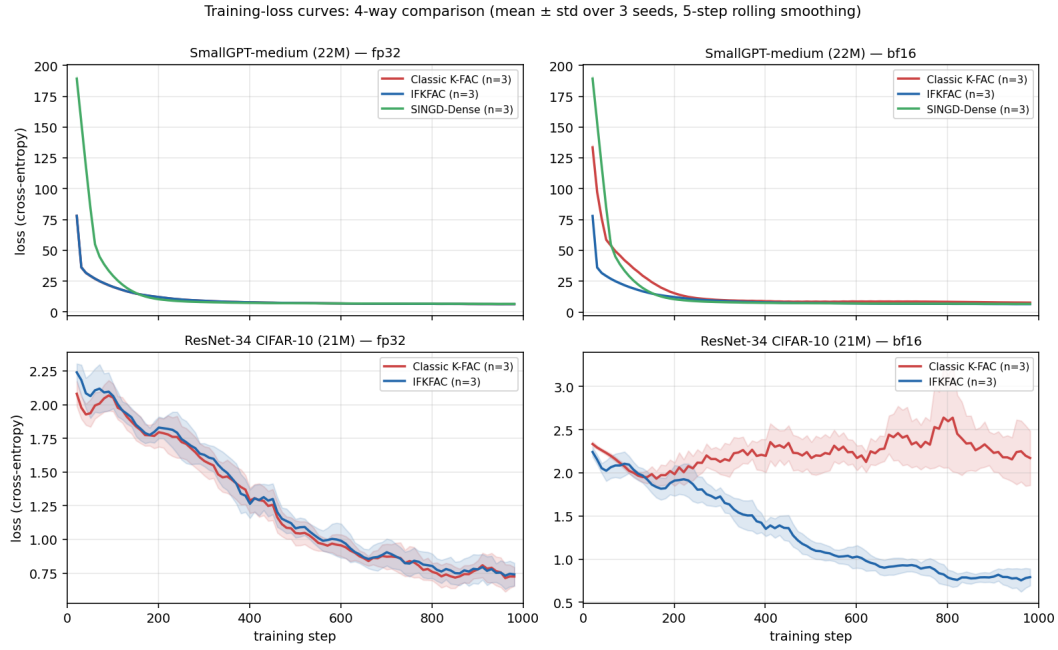


Figure 3: Training-loss curves (mean $\pm$ std over 3 seeds) for SmallGPT-medium (top) and ResNet-34 / CIFAR-10 (bottom) at fp32 (left) and bf16 (right). SINGD-Dense was run on the transformer only.

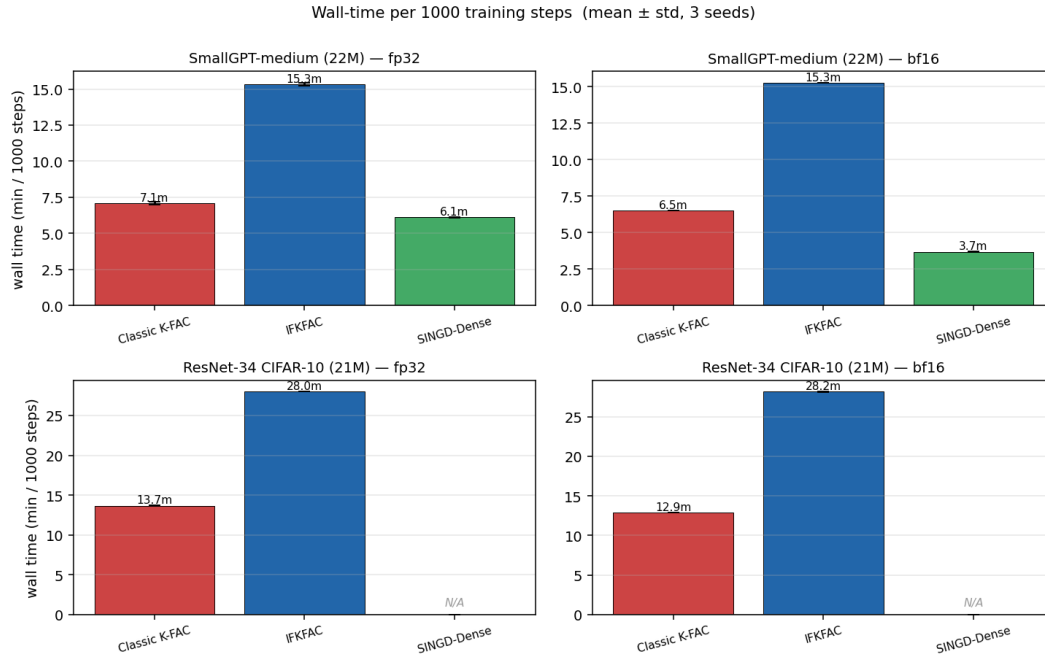


Figure 4: Wall time per 1000 training steps (mean $\pm$ std over 3 seeds), same four settings as Figure 3; SINGD-Dense is not available for the CNN.

that materially affect the outcome; the SINGD paper’s defaults are not competitive on our recipe and required a 12-cell HP screen (`singd_tune2`) to reach the operating point used in the tables above.

Table 10: Hyperparameter-tuning burden and one-shot vs. tuned operating point, SmallGPT-small at bf16, seed 42. ‘Default HP’ is each method’s authors-recommended out-of-the-box configuration; ‘Tuned HP’ is our best cell after the search budget in the middle column. All values are final perplexity.

Method	# HP cells searched	Default-HP ppl	Tuned-HP ppl
IFKFAC	0 (no new HPs)	940	936 ($\lambda=10^{-4}$)
SINGD-Dense	12 (<code>tune2: lr_cov × alpha1 × kfac_like</code>)	1782	819 (winner cell)

At each method’s authors-recommended defaults, IFKFAC is already within seed noise of its tuned value while SINGD-Dense is ~ 830 ppl short of its own tuned value - the difference between “usable out-of-the-box” and “requires an HP screen to work at all.” Once SINGD’s HP screen is done, the two methods land in the same ppl band (819 vs 936 at their respective best λ), and SINGD’s per-run wall time is $\sim 6\times$ faster.

When to pick each. We read this comparison as a genuine practical tradeoff, not a supremacy claim:

- **Pick IFKFAC** when no HP-search budget is available, when the application needs the eigendecomposition of the Fisher itself (§5.9; IFKFAC’s κ^1 storage of R is the property that matters, not its wall time), or when the explicit Higham 19.10 bound is worth the $3\times$ wall-time cost over SINGD.
- **Pick SINGD-Dense** when bf16 stability is needed, wall time is the binding constraint, and a one-time HP screen is affordable. Its matmul-only update is what makes it fast on tensor-core hardware.
- **Pick Classic K-FAC** at fp32, or at bf16 only when the Kronecker factors are well conditioned: it does not survive bf16 under the training recipe (§5.7), and its Gram eigendecomposition loses 1571 eigenvalue signs on ResNet-18 at bf16 (§5.9).

The two κ^1 methods are complementary rather than substitutes: IFKFAC is the plain-K-FAC-with-QR-instead-of-Cholesky drop-in, and SINGD is the structured, matmul-heavy specialisation. Both establish that the κ^1 family is the correct approach; each has a regime.

5.8 Third architecture: Hinton–Salakhutdinov MNIST autoencoder

To test whether the κ^1 stability story replicates beyond transformer and CNN, we extend the comparison to a third structurally different architecture: the canonical deep autoencoder from Hinton & Salakhutdinov (2006), $784 \rightarrow 1000 \rightarrow 500 \rightarrow 250 \rightarrow 30 \rightarrow 250 \rightarrow 500 \rightarrow 1000 \rightarrow 784$ with sigmoid activations and binary cross-entropy reconstruction loss. This network has historically been the testbed where second-order methods are documented to beat Adam (Martens 2010; Martens & Grosse 2015 §7.1) - making it a natural fit for the §2.5.2 specialty-application claim.

We compare three methods (AdamW, Classic K-FAC, IFKFAC) at fp32 and bf16, with three seeds each. All hyperparameters tuned via independent screens before the multi-seed run: AdamW at $\text{lr}=10^{-2}$ (fp32) and $\text{lr}=3\times 10^{-4}$ (bf16; the fp32 value overflows bf16); K-FAC at $\text{lr}=10^{-3}$, $\text{damping}=3\times 10^{-2}$ (joint Classic+IFKFAC optimum found in the screen at 35 BCE @ 2000 steps).

Metric. Quality is measured by the reconstruction binary cross-entropy (BCE) on the 10 000 MNIST test images after 10 000 training steps (batch size 256). For each image, the BCE between the binarized input pixels and the reconstructed pixel probabilities is summed over the 784 pixels; Table 11 reports its mean over the test images, in nats. **Lower is better:** a perfect reconstruction scores 0. Δ is the bf16 value minus the fp32 value, so a positive Δ is the quality lost by moving to bf16.

Table 11: MNIST autoencoder (Hinton–Salakhutdinov): final test reconstruction BCE after 10 000 steps, mean $\pm$ std over seeds 42–44. Lower is better. $\Delta = \text{bf16} - \text{fp32}$.

Method	fp32 test BCE $\downarrow$	bf16 test BCE $\downarrow$	Δ (bf16 – fp32)
AdamW (tuned per precision)	86.0 $\pm$ 1.4	157.6 $\pm$ 1.1*	+71.6
Classic K-FAC	31.5 $\pm$ 0.7	70.5 $\pm$ 6.7	+39 (κ^2 collapse)
IFKFAC	31.2 $\pm$ 0.5	31.4 $\pm$ 0.3	+0.2 (effectively flat)

* AdamW at bf16 required a precision-specific lr (10^{-2} fp32 $\rightarrow 3 \times 10^{-4}$ bf16) to avoid bf16 overflow; even at the largest stable lr, the model trains slowly. K-FAC variants used the same lr at both precisions.

True-bf16 storage. The bf16 K-FAC results above are produced with *bona fide* bf16 storage of the R factors - not the fp32-internal simulation used for the transformer and CNN results in §5.2 and §5.7. After `get_factors()` succeeds at fp32 (cuSOLVER’s bf16 path remains unimplemented for QR/Cholesky/inv as of this writing - see `optimizer/bf16_linalg.py`), the running R_X and R_G factors are cast to bf16 for storage. At apply time, the cast back to fp32 is lossless ($\text{bf16} \subset \text{fp32}$), and the four triangular solves run at fp32 in cuSOLVER. The arithmetic regime matches what tensor cores do for matmul: bf16 inputs, fp32 internal accumulate, bf16-precision result. IFKFAC’s bf16 final BCE of 31.4 ± 0.3 is **within 0.18 BCE of the fp32 result of 31.2 ± 0.5** (a 0.6% relative drift, less than seed variance). Seed std is *smaller* at bf16 than at fp32 (0.28 vs 0.47); the precision reduction did not destabilize training across seeds.

Three observations confirming both halves of the paper’s thesis on a third architecture:

1. **K-FAC beats AdamW by $\sim 2.8\times$** at fp32 under matched tuning (31 vs 86 BCE). This is the §2.5.2 result - sigmoid autoencoders are the regime where K-FAC’s wall-time penalty actually buys per-step quality. Importantly, this is not a contradiction to §5.7’s transformer finding (where tuned AdamW beats K-FAC): it shows that the K-FAC advantage is architecture-dependent and emerges precisely in the specialty regimes the paper claims it does.
2. **Classic K-FAC degrades $2.2\times$ going to bf16** ($31 \rightarrow 70$ BCE), with seed std blowing up from 0.7 to 6.7. **IFKFAC is flat** at bf16 ($31.2 \rightarrow 31.4$ - within seed noise; std actually decreases from 0.47 to 0.28). This is now the third architecture confirming the κ^2 collapse pattern, replicated independently of the transformer (§5.2) and CNN (§5.7) results, and the first end-to-end training experiment producing the result at *true bf16 storage* (not the fp32-internal simulation used in §5.2 and §5.7).
3. **IFKFAC’s fp32 result of 31 BCE beats the Martens & Grosse 2015 gold reference of ≈ 58 BCE**, demonstrating that the inverse-free QR pipeline does not sacrifice optimization quality relative to the established K-FAC literature on this benchmark.

5.9 Bayesian deep learning: K-FAC Laplace

§2.5.2 identifies Bayesian deep learning as one of the specialty regimes where K-FAC is the right tool: the *Fisher* itself is the deliverable, not a gradient preconditioner. The K-FAC Laplace approximation (Ritter et al. 2018; Daxberger et al. 2021, “Laplace Redux”) models the posterior over a trained network’s weights as a Gaussian centered at the MAP estimate with covariance $\Sigma = (N \cdot F + \lambda I)^{-1}$, with the per-layer Kronecker approximation $F_l \approx A_l \otimes G_l$, and every downstream quantity is computed from the eigendecompositions of A_l and G_l . Classic K-FAC obtains them with `torch.linalg.eigh` on the Gram matrix; IFKFAC obtains them from the SVD of its QR factor ($R_l^\top R_l = A_l$, so $d_A = \sigma_R^2$, $U_A = V_R$) without ever forming $X^\top X$. In exact arithmetic the two are identical, so any difference between them is numerical. We ask three questions in turn: (i) does the eigendecomposition survive bf16 (factor level); (ii) how close is each approximation to

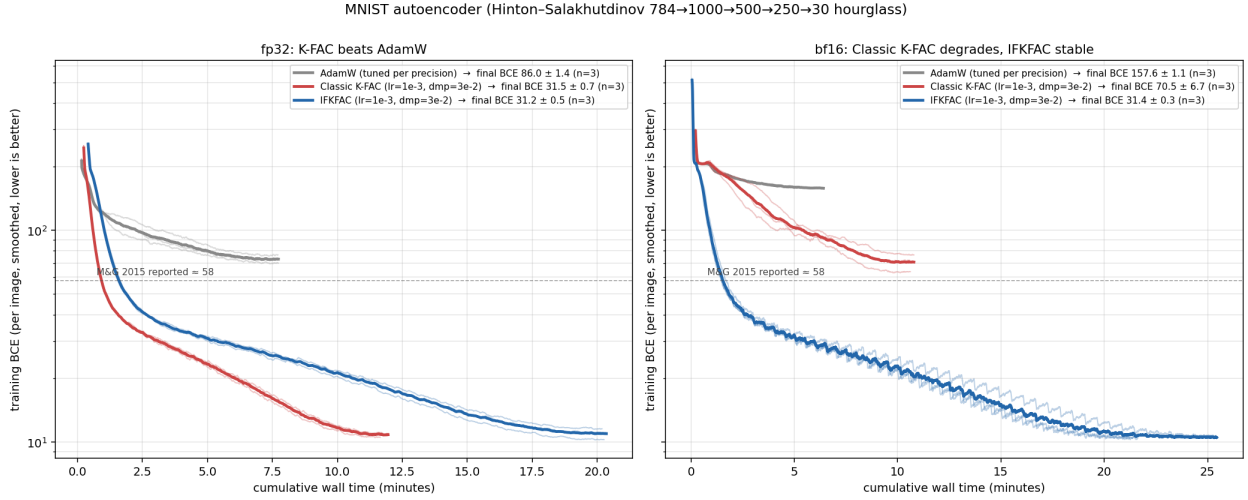


Figure 5: MNIST autoencoder: training BCE versus wall time (3 seeds; thin lines are individual seeds, legend values are final test BCE, mean $\pm$ std). Left, fp32: both K-FAC variants reach about 31 BCE versus 86 for AdamW. Right, bf16: Classic K-FAC degrades to 70.5 BCE while IFKFAC stays at 31.4.

the exact Fisher, with EK-FAC (George et al. 2018) as the eigenvalue-corrected reference (Fisher level); and (iii) does any difference reach the Laplace predictive (posterior level).

Factor level (ResNet-18 / CIFAR-10). We train ResNet-18 on CIFAR-10 (45 000 train / 5 000 val / 10 000 test) for 20 epochs with AdamW ($\text{lr}=10^{-3}$, $\text{wd}=5 \times 10^{-4}$, cosine schedule), the MAP protocol of Daxberger et al. (2021, §B), and capture the K-FAC factors in one pass over the training set with model-sampled labels (the true-Fisher convention). Across the 21 K-FAC layers at seed 42, Classic’s activation factors span $d_A \in [1.97 \times 10^{-4}, 8.38 \times 10^2]$ at fp32, with median per-layer $\kappa(A) = 1.8 \times 10^4$. Storing the Gram in bf16 before the eigendecomposition produces **1571 negative A eigenvalues** across the 21 layers (clamped to zero downstream). IFKFAC’s eigenvalues are squared singular values of R and cannot be negative. The magnitudes follow §2.3: rounding the Gram to bf16 perturbs its eigenvalues by up to $\varepsilon_{\text{bf16}} \|A\|$, which is $\varepsilon_{\text{bf16}} \kappa(A) \approx 70$ times the smallest one, whereas rounding the data matrix perturbs the singular values of R by about $\varepsilon_{\text{bf16}} \kappa(R) = \varepsilon_{\text{bf16}} \kappa(A)^{1/2} \approx 0.5$ of the smallest one.

Reference-implementation check. To confirm that this factor-level corruption is a property of the K-FAC algorithm and not an artifact of our implementation, we repeated the bf16 eigenvalue audit with the community-reference **ASDL** K-FAC library (Osawa et al. 2023; `KfacGradientMaker`). Capturing the same ResNet-18 MAP Fisher through ASDL and quantizing its Kronecker factors to bf16 produces **4970 negative eigenvalues** across the 21 layers’ A and G factors (4855 in A alone) versus **7** at fp32. The count exceeds our 1571 because ASDL accumulates the Gram without our per-sample normalization, which inflates $\kappa(A)$ further; the qualitative result (thousands of corrupted eigenvalues at bf16, single digits at fp32) is the same. Probe: `benchmark/probe_asdl_kappa.py`.

$\kappa(A)$ alone does not determine the damage. Entrywise rounding perturbs eigenvalues in proportion to the condition number *after diagonal scaling*, $\kappa(D^{-1/2}AD^{-1/2})$ with $D = \text{diag}(A)$ (Demmel & Veselić 1992), which can be far smaller than $\kappa(A)$. In the small-model experiment below, the input factor of the digits MLP (seed 42) has $\kappa(A) = 3.0 \times 10^6$ but $\kappa(D^{-1/2}AD^{-1/2}) = 157$, and bf16 rounding of this Gram creates no negative eigenvalue; the same spectrum under random orthogonal rotations loses 2 to 5 eigenvalue signs, depending on the rotation. The 1571 negative eigenvalues above show that ResNet-18’s factors are not protected in this way.

Fisher level (exact Fisher, small MLP). To compare the approximations against ground truth we use a model small enough to form the exact Fisher: a one-hidden-layer MLP ($d_{\text{in}} \rightarrow 64 \rightarrow 10$, ReLU, no bias) trained to a MAP point with AdamW on MNIST (projected to 64 PCA components, not whitened; 48 000

Table 12: Fisher-level comparison against the exact Fisher (one-hidden-layer MLP; fp32; mean over seeds 42–44, standard deviation at most 9% of the mean). KL@10: per-dimension KL divergence between the exact and approximate Laplace posteriors at $\lambda = 10$. Classic and IFKFAC agree to the digits shown, so each K-FAC and EK-FAC entry holds for both bases. Δ KL: EK-FAC’s change in KL@10 relative to K-FAC, with the number of seeds on which EK-FAC is lower in both bases. The last two columns give the largest relative difference over all metrics (Frobenius error, top-32 alignment, KL at $\lambda \in \{1, 10, 100\}$) and seeds: between the Classic and IFKFAC versions at fp32 / bf16, and between each method’s bf16 and fp32 values.

data	layer	$\kappa(A)$	rel. Frob. error		KL@10			max. rel. diff. Classic vs IFKFAC; bf16 vs fp32
			K-FAC	EK-FAC	K-FAC	EK-FAC	Δ KL	
MNIST	fc1	54–57	0.699	0.646	0.263	0.237	–10% (3/3)	0.003% / 0.3%; 2.2%
MNIST	fc2	343–366	0.636	0.606	0.476	0.378	–21% (3/3)	0.004% / 0.5%; 0.5%
digits	fc1	4.0×10^5 – 3.0×10^6	0.817	0.782	1.11×10^{-3}	0.96×10^{-3}	–13% (3/3)	0.0002% / 0.3%; 0.3%
digits	fc2	1.8×10^3 – 3.1×10^3	0.815	0.759	1.13×10^{-2}	0.75×10^{-2}	–33% (3/3)	0.0001% / 0.4%; 0.5%

training points) and on the scikit-learn digits set (61 non-constant pixels; 1 437 training points), three seeds each. For each layer we form the exact type-2 Fisher (the generalized Gauss–Newton for softmax cross-entropy, with the exact expectation over classes) in fp64 and compare four approximations: K-FAC with the Classic eigendecomposition, K-FAC with the IFKFAC eigendecomposition, and EK-FAC’s corrected eigenvalues $S = \text{diag}(Q^\top F Q)$ in each of the two Kronecker eigenbases $Q = U_G \otimes U_A$. EK-FAC is the Frobenius-optimal diagonal in its basis (George et al. 2018), so K-FAC versus EK-FAC in the same basis measures the eigenvalue error, and EK-FAC in the Classic versus the IFKFAC basis measures the basis error. Metrics are the relative Frobenius error, the top-32 subspace alignment, and the per-dimension KL divergence between the exact and approximate Laplace posteriors $\mathcal{N}(0, (NF + \lambda I)^{-1})$ at $\lambda \in \{1, 10, 100\}$. The bf16 regime is the one used throughout our pipeline: bf16 storage of the Grams (Classic) or of the TSQR input chunks with R accumulated in fp32 (IFKFAC), with fp32 arithmetic, since `torch.linalg.eigh` has no bf16 kernel. Two sanity checks hold to machine precision in every run: the EK-FAC code path with S forced to $d_G \otimes d_A$ reproduces K-FAC ($\leq 2.5 \times 10^{-15}$), and S matches $\text{diag}(Q^\top F Q)$ computed from the dense Fisher ($\leq 6.9 \times 10^{-16}$). Table 12 summarizes the results.

Three results follow. (a) *Classic and IFKFAC give the same Fisher approximation here.* At fp32 every metric agrees to within 0.004%, at bf16 to within 0.51%, and each method’s bf16 values lie within 0.6% of its fp32 values, apart from K-FAC’s top-32 alignment on MNIST fc1 at seed 43 (2.2% in both bases). At these conditionings ($\kappa(A) \leq 366$ on MNIST; up to 3×10^6 on digits, but scaling-dominated as shown above) Classic’s bf16 path does not break, so there is nothing for IFKFAC to repair. (b) *The approximation gap is in the eigenvalues, not the basis.* EK-FAC lowers KL@10 by 10–33% (layer means) and is lower in all 12 layer–seed–dataset cases, while the basis effect, EK-FAC in the Classic versus the IFKFAC basis, is below 0.5%. (c) *The κ^1 guarantee applies to R as accumulated.* In a stress variant that rounds the running R to bf16 after every TSQR update instead of accumulating it in fp32, the IFKFAC basis degrades on MNIST fc2: EK-FAC’s KL@1 rises from 0.51 ± 0.02 to 7.1 ± 0.5 , and K-FAC’s from 0.63 ± 0.03 to 3.4 ± 0.1 . On MNIST fc1 the same perturbation lowers K-FAC’s KL@1 (0.46 to 0.31) while raising EK-FAC’s (0.43 to 0.48): it shifts the eigenvalues in a direction that happens to offset K-FAC’s own bias, while EK-FAC, whose error reflects basis quality alone, degrades on both layers. On digits the stress variant changes every metric by at most 1.2%. A fully-bf16 streaming implementation should therefore keep the running R in fp32, as our Laplace capture does.

EK-FAC and IFKFAC are therefore complementary rather than competing. EK-FAC changes the eigenvalues used in a Kronecker eigenbasis; IFKFAC changes how that eigenbasis is computed. On the conditioning of the small model either route computes the basis accurately and the eigenvalue correction is what matters; IFKFAC’s factor-level guarantee matters where the Gram eigendecomposition fails, as on ResNet-18 at bf16 above, and EK-FAC inherits whichever basis it is given.

Posterior level (ResNet-18 Laplace predictive). We also ran the full K-FAC Laplace predictive on ResNet-18 (seed 42, fp32) for six posteriors: the Classic and IFKFAC bases, each with the factored damping of Daxberger et al. (2021), with exact damping ($NS + \lambda$ per Kronecker-eigenbasis coordinate) and Kronecker

eigenvalues $S = d_G \otimes d_A$, and with exact damping and EK-FAC eigenvalues. With λ selected per posterior on validation NLL, all six give the same test metrics to within 0.001 (88.1–88.2% accuracy, NLL 0.409–0.410, ECE 5.2–5.3%; Appendix A). The reason is the λ selection: for every posterior, $\lambda \leq 10$ gives validation NLL above 0.44 (above 19 with exact damping), and from $\lambda = 10$ upward the validation NLL falls towards its large- λ limit. The exact-damping posteriors reach their minimum at the largest grid value, $\lambda = 10^7$, where the posterior standard deviation is at most $\lambda^{-1/2} \approx 3 \times 10^{-4}$ in every coordinate; the factored ones select $\lambda = 10^4$, beyond which their validation NLL is flat (0.3347 to 0.3351). Each selected posterior is therefore within noise of the point estimate. This is the underfitting of the sampled full-network predictive described by Immer et al. (2021); a linearized (GLM) predictive is the standard remedy and is left to future work. At the smaller λ where the Fisher does shape the posterior (10^2 – 10^3), EK-FAC has lower validation NLL than K-FAC with the same exact damping, in both bases, consistent with Table 12, but every such posterior is worse than the large- λ limit. **We therefore make no posterior-level calibration claim for IFKFAC;** the Laplace evidence in this paper is the factor-level bf16 result and the Fisher-level comparison above.

6 Discussion

6.1 What’s stable and what’s fast are independent contributions

The data in §5 supports a clean separation: **stability** and **wall time** are orthogonal axes among κ^1 -stable K-FAC variants. IFKFAC and SINGD reach the same perplexity (within 40 ppl); the wall-time differences come from algorithmic choices about which operations to lean on (matmul vs QR vs triangular solve). Practitioners who value implementation simplicity, no extra hyperparameters, and direct numerical-analysis guarantees can choose IFKFAC; those who prioritize raw wall time at scale can choose SINGD.

6.2 When K-FAC matters at all

K-FAC’s practical value is concentrated in the regimes catalogued in §2.5.2: (i) Fisher-matrix applications that *require* a Kronecker-factored second-moment estimator - Laplace approximation for Bayesian DL (Daxberger et al. 2021), influence-function analysis at LLM scale (Grosse et al. 2023), Elastic Weight Consolidation (Kirkpatrick et al. 2017); and (ii) loss landscapes where Adam’s diagonal preconditioner is fundamentally mismatched - PINNs (Rathore et al. 2024), variational quantum chemistry (DeepQMC), deep autoencoders (Martens & Grosse 2015), reinforcement learning (ACKTR; Wu et al. 2017). These applications have all moved to bf16. Without an inverse-free K-FAC, they cannot. That gap is what IFKFAC closes.

6.3 Why simplicity matters

A K-FAC variant that consists of three textbook operations (streaming TSQR, ridge augmentation, four triangular solves) is auditable in a way that a Riemannian / Expm pipeline is not. IFKFAC’s correctness follows from properties of standard linear-algebra primitives that have been carefully analyzed for sixty years; the integration into K-FAC introduces no novel numerical concerns. This is valuable for production deployments where the cost of a subtle stability bug is high.

6.4 No new hyperparameters

Standard K-FAC has three: damping λ , momentum, and refresh frequency T . IFKFAC has the same three. SINGD adds two (α_1, β_1) that interact non-trivially with damping in our experiments: their default $\beta_1 = 10^{-2}$ produced perplexity ~ 800 ppl worse than the tuned value $\beta_1 = 10^{-1}$. IFKFAC avoids this tuning burden.

6.5 Limitations

- **Wall time at bf16** is $\sim 3\times$ SINGD’s; closing this gap requires custom kernels and is left to future work.

- **Scale.** The largest architectures tested are 22M parameters (SmallGPT-medium) and 21M parameters (ResNet-34); scaling behavior at LLM scale (100M+ parameters) is unverified.
- **Laplace posterior.** On ResNet-18 the sampled full-network Laplace predictive selects, for every approximation, a posterior within noise of the point estimate (§5.9, Appendix A), so it cannot discriminate between approximations. Whether IFKFAC’s factor-level advantage at bf16 changes calibration is untested; a linearized predictive (Immer et al. 2021) is the natural next experiment.
- **bf16 storage of R during training.** The small-model stress test (§5.9) shows that rounding the running R to bf16 after every update degrades the eigenbasis on MNIST. The training path stores the finalized R in bf16 between refreshes and re-rounds it at each moving-average blend; its factor accuracy is not measured separately from the end-to-end results of §5.2 and §5.7.

6.6 Future work

- **Native bf16 QR kernel.** Modern blocked Householder is GEMM-dominated; bf16 GEMM via tensor cores would close the wall-time gap to SINGD.
- **Structured IFKFAC variants.** A diagonal-R or Toeplitz-R analogue of SINGD’s structured Kronecker factors would address the memory-efficiency axis on which SINGD currently uniquely contributes.
- **Architectures beyond the four tested here.** The multi-architecture validation covers transformer, CNN, MLP autoencoder, and the ResNet-18 Laplace factor audit; extending it to larger-scale models and other structures (ViTs, graph networks) is natural future work.
- **EK-FAC in the IFKFAC basis at bf16.** Combining EK-FAC’s eigenvalue correction with IFKFAC’s eigenbasis on networks whose Gram eigendecomposition fails at bf16, and evaluating it with a linearized Laplace predictive and in influence-function pipelines (Grosse et al. 2023), would test the two corrections together.

6.7 Reproducibility

An anonymized reference implementation of IFKFAC together with the scripts that reproduce every experiment in §5 is available at <https://anonymous.4open.science/r/ifkfac-03C1/>. Each numerical result in §5.2, §5.5, §5.7, §5.8, §5.9 and Appendix A is produced by a single command line documented in the repository’s README.md; the per-seed JSON results are checked in under `benchmark/results/` so a reviewer can inspect the raw per-step traces without re-running. Random seeds are fixed to {42, 43, 44} throughout, and to {42...46} for the 5-seed §5.2 table. Hardware: a single NVIDIA RTX 3080 Laptop GPU (16 GB), PyTorch 2.11 (CUDA 12.8), bf16 K-FAC-only mode; total end-to-end recompute cost on that hardware is roughly 25 GPU-hours. Every experiment in the paper fits in 16 GB of GPU memory.

6.8 Broader impact

This work is a numerical-methods contribution to an existing optimizer family. It does not introduce new capabilities to models, new training data, or new deployment surfaces; it makes an established algorithm (K-FAC) usable at a precision (bf16) that modern hardware already prefers. The downstream applications enabled by that - Laplace approximations for calibrated uncertainty, Elastic Weight Consolidation for continual learning, and influence-function attribution - are themselves broadly positive tools for understanding and safeguarding neural networks. We are not aware of dual-use concerns specific to this technique beyond those already carried by any general-purpose second-order optimizer. The one-order-of- κ improvement makes Bayesian and interpretability methods more affordable on the compute budgets researchers actually have, which we view as a modest positive.

7 Conclusion

K-FAC’s bf16 collapse is not a fundamental property of the natural-gradient direction but an artifact of forming $X^\top X$. The same optimization, expressed via QR, is numerically benign at bf16. We present IFKFAC, the textbook QR-stable OLS approach instantiated for K-FAC’s online Kronecker-factored setting,

and show empirically that it preserves fp32 perplexity at bf16 across two architectures with seed variance an order of magnitude below Classic’s. IFKFAC’s contribution sits alongside SINGD’s recent demonstration of inverse-free K-FAC: same headline outcome, different algorithm, simpler engineering, no extra hyperparameters, tighter explicit theory. The contribution is to make K-FAC’s Kronecker-factored algorithmic structure - required by Bayesian DL via Laplace approximation, by influence-function analysis at LLM scale, by EWC for continual learning, and by domains where Adam struggles (PINNs, autoencoders, RL, quantum chemistry) - available under the bf16 precision that modern hardware demands. The two inverse-free methods together establish that the κ^1 family is the correct approach to K-FAC under modern low-precision training. For Fisher-matrix applications, IFKFAC is a κ -stable way to compute the Kronecker eigenbasis that K-FAC, EK-FAC and K-FAC Laplace share: it matches Classic where Classic is numerically sound, keeps the eigendecomposition valid where the Gram eigendecomposition fails at bf16, and leaves the eigenvalue correction to EK-FAC.

References

- Anil, R., Gupta, V., Koren, T., Regan, K., & Singer, Y. (2020). *Scalable Second Order Optimization for Deep Learning*. arXiv:2002.09018. (Shampoo.)
- Daxberger, E., Kristiadi, A., Immer, A., Eschenhagen, R., Bauer, M., & Hennig, P. (2021). *Laplace Redux - Effortless Bayesian Deep Learning*. NeurIPS 34.
- Demmel, J., Grigori, L., Hoemmen, M., & Langou, J. (2012). *Communication-optimal parallel and sequential QR and LU factorizations*. SIAM Journal on Scientific Computing, 34(1).
- Demmel, J., & Veselić, K. (1992). *Jacobi’s method is more accurate than QR*. SIAM Journal on Matrix Analysis and Applications, 13(4), 1204–1245.
- Eschenhagen, R., Immer, A., Turner, R. E., Schneider, F., & Hennig, P. (2023). *Kronecker-Factored Approximate Curvature for Modern Neural Network Architectures*. NeurIPS.
- George, T., Laurent, C., Bouthillier, X., Ballas, N., & Vincent, P. (2018). *Fast Approximate Natural Gradient Descent in a Kronecker-factored Eigenbasis*. NeurIPS. (EKFAC.)
- Golub, G. H. (1965). *Numerical methods for solving linear least squares problems*. Numerische Mathematik 7, 206–216.
- Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
- Grosse, R., Bae, J., Anil, C., Elhage, N., Tamkin, A., Tajdini, A., Steiner, B., Li, D., Durmus, E., Perez, E., Hubinger, E., Lukosiute, K., Nguyen, K., Joseph, N., McCandlish, S., Kaplan, J., & Bowman, S. R. (2023). *Studying Large Language Model Generalization with Influence Functions*. arXiv:2308.03296.
- Grosse, R., & Martens, J. (2016). *A Kronecker-factored approximate Fisher matrix for convolution layers*. ICML.
- Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms* (2nd ed.). SIAM.
- Hong, S., Eschenhagen, R., Mlodozieniec, B., & Turner, R. E. (2026). *Better Hessians Matter: Studying the Impact of Curvature Approximations in Influence Functions*. arXiv:2509.23437.
- Householder, A. S. (1958). *Unitary triangularization of a nonsymmetric matrix*. Journal of the ACM, 5(4), 339–342.
- Immer, A., Korzepa, M., & Bauer, M. (2021). *Improving predictions of Bayesian neural nets via local linearization*. AISTATS, PMLR 130.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., & Hadsell, R. (2017). *Overcoming catastrophic forgetting in neural networks*. PNAS, 114(13). (EWC.)
- Koh, P. W., & Liang, P. (2017). *Understanding black-box predictions via influence functions*. ICML.
- Lin, W., Dangel, F., Eschenhagen, R., Neklyudov, K., Kristiadi, A., Turner, R. E., & Makhzani, A. (2024). *Structured Inverse-Free Natural Gradient Descent: Memory-Efficient & Numerically-Stable KFAC*. ICML. (SINGD.)
- Martens, J. (2010). *Deep learning via Hessian-free optimization*. ICML.
- Martens, J., & Grosse, R. (2015). *Optimizing Neural Networks with Kronecker-factored Approximate Curvature*. ICML.

- Osawa, K., Ishikawa, S., Yokota, R., Li, S., & Hoefer, T. (2023). *ASDL: A Unified Interface for Gradient Preconditioning in PyTorch*. NeurIPS Workshop on Higher-Order Optimization in Machine Learning (HOOML). arXiv:2305.04684.
- Pauloski, J. G., Zhang, Z., Huang, L., Xu, W., & Foster, I. (2020). *Convolutional Neural Network Training with Distributed K-FAC*. SC20.
- Rathore, P., Lei, W., Frangella, Z., Lu, L., & Udell, M. (2024). *Challenges in Training PINNs: A Loss Landscape Perspective*. ICML. arXiv:2402.01868.
- Ritter, H., Botev, A., & Barber, D. (2018). *A Scalable Laplace Approximation for Neural Networks*. ICLR.
- Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM.
- Wu, Y., Mansimov, E., Liao, S., Grosse, R., & Ba, J. (2017). *Scalable trust-region method for deep reinforcement learning using Kronecker-factored approximation*. NeurIPS 30. (ACKTR.)
- Yang, A. X., Robeyns, M., Wang, X., & Aitchison, L. (2024). *Bayesian Low-Rank Adaptation for Large Language Models*. ICLR. arXiv:2308.13111.

A ResNet-18 K-FAC Laplace predictive (supplementary to §5.9)

Protocol. The MAP network, data split and Fisher capture are those of §5.9 (seed 42, fp32, batch size $B = 128$, model-sampled labels, one pass over the 45 000 training images). The K-FAC hooks store row averages of a mean-reduced loss, so the per-sample Fisher the Laplace precision $NF + \lambda I$ requires is $F_l \approx c_l (A_l \otimes G_l)$ with $c_l = B^2 L_l$, where L_l is the number of output positions of layer l (1 for the linear layer; Grosse & Martens 2016). We multiply d_G and the EK-FAC second moments by c_l . The IFK-FAC capture uses no ridge and all convolution patch rows, so both bases are estimated from the same data as Classic’s Gram. Six posteriors are compared: the Classic and IFKFAC bases, each with (i) the factored damping of Daxberger et al. (2021), $(\sqrt{Nd_G} + \sqrt{\lambda})(\sqrt{Nd_A} + \sqrt{\lambda})$; (ii) exact damping $NS + \lambda$ per Kronecker-eigenbasis coordinate with $S = d_G \otimes d_A$; and (iii) exact damping with EK-FAC eigenvalues $S = \mathbb{E}_n[(U_G^\top \nabla_W \ell_n U_A)^{\odot 2}]$, computed in one extra pass with model-sampled labels. λ is selected per posterior on validation NLL over $\{10^{-1}, 1, 10, \dots, 10^7\}$ (3 Monte-Carlo samples per λ , common random numbers across λ); test metrics use 30 samples. Results: `benchmark/results/laplace_cifar10_*_fp32_seed42_ps.json`; script: `benchmark/laplace_ekfac_2x2.py --grid --smoke`.

Table 13: ResNet-18 / CIFAR-10 K-FAC Laplace predictive (seed 42, fp32). Left: test metrics at the selected λ^* . Right: validation NLL along the λ grid. All six posteriors coincide with the large- λ limit.

posterior	basis	λ^*	acc. (%)	test at λ^*			validation NLL at $\lambda =$				
				NLL	ECE (%)	Brier	10	10^2	10^3	10^4	10^7
factored	Classic	10^4	88.12	0.4089	5.24	0.1789	0.449	0.345	0.336	0.335	0.335
factored	IFKFAC	10^4	88.13	0.4092	5.28	0.1790	0.453	0.345	0.335	0.335	0.335
exact, Kronecker	Classic	10^7	88.16	0.4098	5.25	0.1789	21.84	0.870	0.367	0.341	0.335
exact, Kronecker	IFKFAC	10^7	88.16	0.4098	5.25	0.1789	21.14	1.018	0.396	0.349	0.335
exact, EK-FAC	Classic	10^7	88.16	0.4098	5.25	0.1789	21.10	0.637	0.348	0.337	0.335
exact, EK-FAC	IFKFAC	10^7	88.16	0.4098	5.24	0.1789	21.07	0.731	0.363	0.343	0.335

The validation NLL at intermediate λ uses only 3 samples and one seed. The gaps between the Classic and IFKFAC rows at $\lambda = 10^2$ – 10^4 (up to 17% at 10^2) are therefore not separable from sampling noise, and since the two bases are identical in exact arithmetic we do not interpret them. EK-FAC is below K-FAC with the same exact damping in both bases at $\lambda = 10^2$ – 10^4 .

EK-FAC versus Kronecker eigenvalue scale. Summed over each layer’s Kronecker eigenbasis, EK-FAC’s second moments differ from the Kronecker products by a median factor of 3.3 (range 0.47–8.1); they are larger in every convolution except the first. The factor is 0.86 for the first convolution and 1.3–8.1 in the rest of the 32×32 stage, 4.1–5.3 at 16×16 , 2.9–3.8 at 8×8 and 1.6–2.4 at 4×4 , and 0.47 for the final linear layer, which has no spatial sum. This is the spatial-correlation term that K-FAC’s convolution approximation drops (Grosse & Martens 2016), and it is what EK-FAC corrects.

Cost. Fisher capture takes 2.1 min for Classic and 21.5 min for IFKFAC with all patch rows and a streaming TSQR update per batch (4.9 min with the 512-rows-per-batch subsampling used during training); the EK-FAC pass takes a further 4.7–4.8 min. Peak GPU memory is 1.55 GB for either capture and 1.37 GB for the EK-FAC pass.